

Problem Statement

A library tracks daily book borrowing logs for its members. Each log entry includes the member ID, book section, date, and return status such as Returned, Pending, or Overdue. Your task is to perform analysis over this borrowing data to identify return rates, overdue patterns, and section-wise summaries.

Declare Class

```
class LibraryBorrowingAnalyzer:
```

No need to use init method.

If needed, follow recommendations:

```
def __init__(self):  
    pass
```

Operations

1. Create Library Borrowing DataFrame

Converts raw library borrowing logs into a structured Pandas DataFrame.

Function Prototype:

```
def create_borrowing_df(self, data: list) -> pd.DataFrame:
```

Example Input:

```
create_borrowing_df([  
    [301, "Fiction", "2024-07-01", "Returned"],  
    [302, "Science", "2024-07-01", "Overdue"],
```

```
[301, "Fiction", "2024-07-02", "Pending"]
])
```

Expected Output:

	MemberID	Section	Date	ReturnStatus
0	301	Fiction	2024-07-01	Returned
1	302	Science	2024-07-01	Overdue
2	301	Fiction	2024-07-02	Pending

Implementation Flow:

💡 Create a DataFrame using columns: ["MemberID", "Section", "Date", "ReturnStatus"].

💡 Ensure Date remains as a string. Do not convert it to datetime.

✓ 2. Calculate Monthly Return Rate per Member

Returns a DataFrame showing percentage of borrowed books returned per member per month.

Function Prototype:

```
def compute_monthly_return_rate(self, df: pd.DataFrame)
-> pd.DataFrame:
```

Example Input:

Assuming records from July 2024.

Expected Output:

MemberID	Month	Return Rate
----------	-------	-------------

0	301	2024-07	50.0
1	302	2024-07	0.0

Implementation Flow:

💡 Extract "Year-Month" from Date using `df["Date"].str[:7]`. This will remove the day and keep year-month intact.

💡 Group by `MemberID` and `Month`, then count total borrowed records.

💡 Create a new DataFrame by applying filter over `ReturnStatus == "Returned"`.

💡 Group the returned DataFrame by `MemberID` and `Month`, then count returned records.

💡 Merge original total count DataFrame and returned count DataFrame.

💡 Compute `Return Rate = (Returned / Total) * 100`.

💡 Replace missing returned counts with 0 before calculating percentage.

=====Breakdown=====

```
DF_orginal=["MemberID", "Section", "Date",  
"ReturnStatus"].
```

```
DF_month_df=["MemberID", "Section", "Date",  
"ReturnStatus", "Month"].
```

#table1

Total_df= [{"MemberID", "Month", "Total"}].

#Table 2 Returned

#filtered

Filter_returned_df =[MemberID", "Month",
"ReturnedTotal"]

=====solution.py=====

```
import pandas as pd
```

```
class LibraryBorrowingAnalyzer:
```

```
    def create_borrowing_df(self, data: list) -> pd.DataFrame:
```

```
        df=pd.DataFrame(data,columns=["MemberID", "Section", "Date",  
"ReturnStatus"])
```

```
        return df
```

```
    def compute_monthly_return_rate(self, df: pd.DataFrame) ->  
pd.DataFrame:
```

```
        df=df.copy()
```

```
        df['Month']=df['Date'].str[:7]
```

```
        #print(df)
```

```
        #print(df.columns)
```

```
        # select memberid,month,count(returnstatus) as total from lib  
where Returnstatus='Returned'
```

```
        # group by memberid,month;
```

```

# Table 1 total return status of all

total_df=df.groupby(['MemberID','Month']).size().reset_index(name='Total')

print(total_df)

# table 2 total returned count

filtered_df=df[df['ReturnStatus']=='Returned']

filtered_return_count_df=filtered_df.groupby(['MemberID','Month']).size()
().reset_index(name='Returned')

print(filtered_return_count_df)

joined_df= pd.merge(total_df, filtered_return_count_df,
on=["MemberID", "Month"], how="left")

```

✓ 3. Add Overdue Flag Column

Adds a column "IsOverdue" with 1 if return status is "Overdue", else 0.

Function Prototype:

```
def add_overdue_flag(self, df: pd.DataFrame) ->
pd.DataFrame:
```

Example Input:

```
add_overdue_flag(df)
```

Expected Output:

	MemberID	Section	Date	ReturnStatus	IsOverdue
0	301	Fiction	2024-07-01	Returned	0
1	302	Science	2024-07-01	Overdue	1
2	301	Fiction	2024-07-02	Pending	0

Implementation Flow:

💡 Use `df["ReturnStatus"] == "Overdue"`.

💡 Convert the boolean result to integer using `.astype(int)`.

💡 Store the result in a new column named `"IsOverdue"`.

✅ 4. Filter Members with High Overdue Count

Returns a DataFrame of members with more than `n` overdue records.

Function Prototype:

```
def high_overdue_members(self, df: pd.DataFrame,
threshold: int) -> pd.DataFrame:
```

Example Input:

```
high_overdue_members(df, 1)
```

Expected Output:

	MemberID	Overdue Count
0	302	3
1	305	4

Implementation Flow:

- 💡 Filter rows where **ReturnStatus** is "Overdue".
 - 💡 Group by **MemberID** and count overdue records.
 - 💡 Rename the count column as "**Overdue Count**".
 - 💡 Return only those members where count is greater than **threshold**.
-

✅ 5. Section-wise Return Status Summary

Returns counts of each return status per library section.

Function Prototype:

```
def section_return_summary(self, df: pd.DataFrame) -> pd.DataFrame:
```

Example Input:

```
section_return_summary(df)
```

Expected Output:

	Section	Returned	Pending	Overdue
0	Fiction	3	2	1

1 Science 1 0 2

Implementation Flow:

💡 Implement `pd.crosstab` using "Section" and "ReturnStatus" in the input DataFrame.

💡 Set column name to `None`.

💡 Use `reindex` with column argument as `["Returned", "Pending", "Overdue"]`.

💡 Replace null values with `0`.

💡 Use `reset_index()` and return the resultant DataFrame.

LibraryBorrowingAnalyzer Driver Code

```
import pandas as pd
```

```
i
```

```
# Trainees need to write only the class and method implementations.
```

```
# class LibraryBorrowingAnalyzer:
```

```
#     def create_borrowing_df(self, data: list) -> pd.DataFrame:
```

```
#         pass
```

```
#
```

```

# def compute_monthly_return_rate(self, df:
pd.DataFrame) -> pd.DataFrame:

#     pass

#

# def add_overdue_flag(self, df: pd.DataFrame) ->
pd.DataFrame:

#     pass

#

# def high_overdue_members(self, df: pd.DataFrame,
threshold: int) -> pd.DataFrame:

#     pass

#

# def section_return_summary(self, df: pd.DataFrame) ->
pd.DataFrame:

#     pass


if __name__ == "__main__":
    analyzer = LibraryBorrowingAnalyzer()

    data = [
        [301, "Fiction", "2024-07-01", "Returned"],

```

```
[302, "Science", "2024-07-01", "Overdue"],  
[301, "Fiction", "2024-07-02", "Pending"],  
[302, "Science", "2024-07-02", "Overdue"],  
[303, "History", "2024-07-01", "Returned"],  
[303, "History", "2024-07-03", "Overdue"],  
[304, "Fiction", "2024-07-03", "Returned"],  
[305, "Science", "2024-07-04", "Overdue"],  
[305, "Science", "2024-07-05", "Overdue"],  
[301, "Fiction", "2024-07-06", "Returned"],  
[304, "Fiction", "2024-07-07", "Pending"],  
[302, "Science", "2024-07-08", "Overdue"]  
]
```

```
df = analyzer.create_borrowing_df(data)  
print("Library Borrowing DataFrame:")  
print(df)  
  
monthly_rate =  
analyzer.compute_monthly_return_rate(df)  
print("\nMonthly Return Rate:")  
print(monthly_rate)
```

```
flagged_df = analyzer.add_overdue_flag(df)
print("\nBorrowing DataFrame with Overdue Flag:")
print(flagged_df)

high_overdue = analyzer.high_overdue_members(df, 1)
print("\nHigh Overdue Members:")
print(high_overdue)

summary = analyzer.section_return_summary(df)
print("\nSection-wise Return Status Summary:")
print(summary)
```

=====

Q2 Food delivery order analysis.

Problem Statement

A food delivery platform tracks customer orders placed across different restaurants. Each order entry includes order ID, customer ID, restaurant name, order date, order amount, and order status. Your task is to clean, organize,

and analyze this order data to identify duplicate orders, missing values, customer spending, and restaurant-wise order trends.

✓ **DeclareClass**

```
class FoodDeliveryOrderAnalyzer:
```

No need to use init method.

If needed, follow recommendations:

```
def __init__(self):  
    pass
```

Operations

✓ **1. Create Food Delivery Orders DataFrame**

Converts raw food delivery order records into a structured Pandas DataFrame.

Function Prototype:

```
def create_orders_df(self, data: list) -> pd.DataFrame:
```

Example Input:

```
create_orders_df([  
    [501, 1001, "SpiceHub", "2024-08-01", 450.0,  
    "Delivered"],  
    [502, 1002, "BurgerTown", "2024-08-01", None,  
    "Cancelled"],  
    [501, 1001, "SpiceHub", "2024-08-01", 450.0, "Delivered"]  
])
```

Expected Output:

	OrderID	CustomerID	Restaurant	Date	Amount	Status
0	501	1001	SpiceHub	2024-08-01	450.0	Delivered
1	502	1002	BurgerTown	2024-08-01	NaN	Cancelled
2	501	1001	SpiceHub	2024-08-01	450.0	Delivered

Implementation Flow:

💡 Create a DataFrame using columns: ["OrderID", "CustomerID", "Restaurant", "Date", "Amount", "Status"].

💡 Ensure Date remains as a string.

💡 Do not remove duplicate rows in this function.

✅ 2. Remove Duplicate Orders

Returns a DataFrame after removing duplicate order records based on **OrderID**.

Function Prototype:

```
def remove_duplicate_orders(self, df: pd.DataFrame) -> pd.DataFrame:
```

Example Input:

```
remove_duplicate_orders(df)
```

Expected Output:

	OrderID	CustomerID	Restaurant	Date	Amount	Status
0	501	1001	SpiceHub	2024-08-01	450.0	Delivered
1	502	1002	BurgerTown	2024-08-01	NaN	Cancelled

Implementation Flow:

- 💡 Use `drop_duplicates()` on the DataFrame.
 - 💡 Apply duplicate removal using `subset=["OrderID"]`.
 - 💡 Keep the first occurrence of each duplicate order.
 - 💡 Reset the index after removing duplicates.
-

✅ 3. Fill Missing Order Amounts

Replaces missing order amount values with 0.

Function Prototype:

```
def fill_missing_amounts(self, df: pd.DataFrame) ->
pd.DataFrame:
```

Example Input:

```
fill_missing_amounts(df)
```

Expected Output:

	OrderID	CustomerID	Restaurant	Date	Amount	Status
--	---------	------------	------------	------	--------	--------

0	501	1001	SpiceHub	2024-08-01	450.0
Delivered					
1	502	1002	BurgerTown	2024-08-01	0.0
Cancelled					
2	503	1003	PizzaZone	2024-08-02	700.0
Delivered					

Implementation Flow:

- 💡 Use `fillna()` on the "Amount" column.
 - 💡 Replace missing values with 0.
 - 💡 Store the cleaned values back in the "Amount" column.
 - 💡 Return the updated DataFrame.
-

✅ 4. Calculate Total Spending per Customer

Returns total delivered order amount spent by each customer.

Function Prototype:

```
def customer_total_spending(self, df: pd.DataFrame) ->
pd.DataFrame:
```

Example Input:

```
customer_total_spending(df)
```

Expected Output:

	CustomerID	Total Spending
0	1001	900.0

1 1003 700.0

Implementation Flow:

- 💡 Filter rows where **Status** is "Delivered".
 - 💡 Group by **CustomerID**.
 - 💡 Use aggregation function **sum()** on "Amount".
 - 💡 Rename the amount column as "**Total Spending**".
 - 💡 Reset index and return the resultant DataFrame.
-

✅ 5. Restaurant-wise Status Pivot Summary

Returns restaurant-wise count of orders for each order status.

Function Prototype:

```
def restaurant_status_summary(self, df: pd.DataFrame) -> pd.DataFrame:
```

Example Input:

```
restaurant_status_summary(df)
```

Expected Output:

	Restaurant	Cancelled	Delivered	Preparing
0	BurgerTown	2	0	1
1	PizzaZone	0	3	0
2	SpiceHub	1	4	0

Implementation Flow:

- 💡 Use `pivot_table()` on the input DataFrame.
- 💡 Set `index="Restaurant"`.
- 💡 Set `columns="Status"`.
- 💡 Set `values="OrderID"`.
- 💡 Use aggregation function as count.
- 💡 Replace missing values with `0`.
- 💡 Reset index before returning the final DataFrame.

FoodDeliveryOrderAnalyzer Driver Code

```
import pandas as pd
```

```
# Trainee will write this class in solution.py
```

```
# class FoodDeliveryOrderAnalyzer:
```

```
#     def create_orders_df(self, data: list) -> pd.DataFrame:
```

```
#         pass
```

```
#
```

```
#     def remove_duplicate_orders(self, df: pd.DataFrame)  
-> pd.DataFrame:
```

```
#         pass
```

```
#

# def fill_missing_amounts(self, df: pd.DataFrame) ->
pd.DataFrame:

#     pass

#

# def customer_total_spending(self, df: pd.DataFrame) ->
pd.DataFrame:

#     pass

#

# def restaurant_status_summary(self, df:
pd.DataFrame) -> pd.DataFrame:

#     pass


if __name__ == "__main__":
    analyzer = FoodDeliveryOrderAnalyzer()

    order_data = [
        [501, 1001, "SpiceHub", "2024-08-01", 450.0,
"Delivered"],
        [502, 1002, "BurgerTown", "2024-08-01", None,
"Cancelled"],
```

```
[501, 1001, "SpiceHub", "2024-08-01", 450.0,
"Delivered"],

[503, 1003, "PizzaZone", "2024-08-02", 700.0,
"Delivered"],

[504, 1001, "SpiceHub", "2024-08-03", 450.0,
"Delivered"],

[505, 1004, "BurgerTown", "2024-08-03", 300.0,
"Preparing"],

[506, 1005, "PizzaZone", "2024-08-04", None,
"Delivered"],

[507, 1002, "BurgerTown", "2024-08-04", 250.0,
"Cancelled"]

]
```

```
df = analyzer.create_orders_df(order_data)
print("Orders DataFrame:")
print(df)
```

```
unique_df = analyzer.remove_duplicate_orders(df)
print("\nOrders After Removing Duplicates:")
print(unique_df)
```

```
filled_df = analyzer.fill_missing_amounts(unique_df)
print("\nOrders After Filling Missing Amounts:")
print(filled_df)
```

```
spending = analyzer.customer_total_spending(filled_df)
print("\nCustomer Total Spending:")
print(spending)
```

```
summary =
analyzer.restaurant_status_summary(filled_df)
print("\nRestaurant Status Summary:")
print(summary)
```

online learning course progress analysis.

Problem Statement

An online learning platform tracks course progress records for students. Each record includes student ID, course name, enrollment date, completed lessons, total lessons, and course status. Your task is to transform and analyze this course progress data to calculate completion percentage, classify learner progress, sort records, and generate course-level summaries.

✓ DeclareClass

```
class CourseProgressAnalyzer:
```

No need to use init method.

If needed, follow recommendations:

```
def __init__(self):
```

```
    pass
```

📌 Operations

✓ 1. Create Course Progress DataFrame

Converts raw course progress records into a structured Pandas DataFrame.

Function Prototype:

```
def create_progress_df(self, data: list) -> pd.DataFrame:
```

Example Input:

```
create_progress_df([
    [701, "Python Basics", "2024-09-01", 8, 10, "Active"],
    [702, "Data Analysis", "2024-09-02", 15, 15, "Completed"],
    [703, "Python Basics", "2024-09-03", 2, 10, "Active"]
])
```

Expected Output:

StudentID	Course	EnrollmentDate	CompletedLessons	TotalLessons	Status
-----------	--------	----------------	------------------	--------------	--------

0	701	Python Basics	2024-09-01	8
10		Active		
1	702	Data Analysis	2024-09-02	15
15		Completed		
2	703	Python Basics	2024-09-03	2
10		Active		

Implementation Flow:

💡 Create a DataFrame using columns: ["StudentID", "Course", "EnrollmentDate", "CompletedLessons", "TotalLessons", "Status"].

💡 Ensure `EnrollmentDate` remains as a string.

💡 Do not calculate completion percentage in this function.

✅ 2. Calculate Completion Percentage

Adds a "`Completion Percentage`" column based on completed lessons and total lessons.

Function Prototype:

```
def add_completion_percentage(self, df: pd.DataFrame) -> pd.DataFrame:
```

Example Input:

```
add_completion_percentage(df)
```

Expected Output:

	StudentID	Course	EnrollmentDate	CompletedLessons	TotalLessons	Status	Completion Percentage
0	701	Python Basics	2024-09-01	8			
10		Active			80.0		
1	702	Data Analysis	2024-09-02	15			
15		Completed			100.0		
2	703	Python Basics	2024-09-03	2			
10		Active			20.0		

Implementation Flow:

💡 Use `assign()` to create a new column named "Completion Percentage".

💡 Formula: $(\text{CompletedLessons} / \text{TotalLessons}) * 100$.

💡 Round the calculated percentage to 2 decimal places.

💡 Return the updated DataFrame.

✅ 3. Classify Learner Progress

Adds a "ProgressLevel" column based on completion percentage.

Function Prototype:

```
def classify_progress_level(self, df: pd.DataFrame) ->
pd.DataFrame:
```

Example Input:

```
classify_progress_level(df)
```

Expected Output:

	StudentID	Course	Completion Percentage	ProgressLevel
0	701	Python Basics	80.0	High
1	702	Data Analysis	100.0	High
2	703	Python Basics	20.0	Low

Implementation Flow:

💡 Use `apply()` on "Completion Percentage".

💡 If completion percentage is greater than or equal to 75, classify as "High".

💡 If completion percentage is greater than or equal to 40 and less than 75, classify as "Medium".

💡 Otherwise classify as "Low".

💡 Store the result in a new column named "ProgressLevel".

✅ 4. Sort Students by Completion Percentage

Returns records sorted by completion percentage in descending order.

Function Prototype:

```
def sort_by_completion(self, df: pd.DataFrame) -> pd.DataFrame:
```

Example Input:

```
sort_by_completion(df)
```

Expected Output:

	StudentID	Course	Completion Percentage
0	702	Data Analysis	100.0
1	701	Python Basics	80.0
2	703	Python Basics	20.0

Implementation Flow:

- 💡 Use `sort_values()`.
- 💡 Sort by "`Completion Percentage`".
- 💡 Set `ascending=False`.
- 💡 Reset index after sorting.
- 💡 Return the sorted DataFrame.

✅ 5. Course-wise Progress Summary

Returns average completion percentage and student count for each course.

Function Prototype:

```
def course_progress_summary(self, df: pd.DataFrame) -> pd.DataFrame:
```

Example Input:

```
course_progress_summary(df)
```

Expected Output:

	Course	Average Completion	Student Count
0	Data Analysis	100.00	1
1	Python Basics	50.00	2

Implementation Flow:

💡 Group by "Course".

💡 Use `agg()` to calculate mean of "Completion Percentage" and count of "StudentID".

💡 Rename the output columns as "Average Completion" and "Student Count".

💡 Round "Average Completion" to 2 decimal places.

💡 Reset index and return the resultant DataFrame.

CourseProgressAnalyzer Driver Code

```
import pandas as pd
```

```
# Trainee will write this class in solution.py
```

```
# class CourseProgressAnalyzer:
```

```
#     def create_progress_df(self, data: list) ->  
pd.DataFrame:
```

```
#         pass
```

```
#
```

```
#     def add_completion_percentage(self, df:  
pd.DataFrame) -> pd.DataFrame:
```

```
#         pass
```

```
#
```

```
#     def classify_progress_level(self, df: pd.DataFrame) ->  
pd.DataFrame:
```

```
#         pass
```

```
#
```

```
#     def sort_by_completion(self, df: pd.DataFrame) ->  
pd.DataFrame:
```

```
#         pass
```

```
#  
  
# def course_progress_summary(self, df: pd.DataFrame)  
-> pd.DataFrame:  
  
#     pass
```

```
if __name__ == "__main__":
```

```
    analyzer = CourseProgressAnalyzer()
```

```
    progress_data = [  
        [701, "Python Basics", "2024-09-01", 8, 10, "Active"],  
        [702, "Data Analysis", "2024-09-02", 15, 15,  
"Completed"],  
        [703, "Python Basics", "2024-09-03", 2, 10, "Active"],  
        [704, "Machine Learning", "2024-09-04", 12, 20,  
"Active"],  
        [705, "Data Analysis", "2024-09-05", 10, 15, "Active"],  
        [706, "Machine Learning", "2024-09-06", 20, 20,  
"Completed"]  
    ]
```

```
    df = analyzer.create_progress_df(progress_data)
```

```
print("Course Progress DataFrame:")
```

```
print(df)
```

```
percentage_df = analyzer.add_completion_percentage(df)
```

```
print("\nCourse Progress with Completion Percentage:")
```

```
print(percentage_df)
```

```
classified_df =
```

```
analyzer.classify_progress_level(percentage_df)
```

```
print("\nCourse Progress with Progress Level:")
```

```
print(classified_df)
```

```
sorted_df = analyzer.sort_by_completion(classified_df)
```

```
print("\nStudents Sorted by Completion:")
```

```
print(sorted_df)
```

```
summary =
```

```
analyzer.course_progress_summary(classified_df)
```

```
print("\nCourse Progress Summary:")
```

```
print(summary)
```

